

Application Note 001

Vayyar API Reference Guide

Revision 6

Vayyar Imaging Ltd.

Web: https://vayyar.com

Notice

Copyright 2020 Vayyar Imaging Ltd. All rights reserved.

This document is the property of Vayyar and all materials and information contained herein are the confidential and proprietary information of Vayyar and are protected by international copyright laws. The copyrights are owned by Vayyar or the original creator of the material. The materials and information contained herein are provided to you for internal information purposes only, and no portion of the materials and information herein may be copied, reproduced, distributed, transmitted, displayed, published, broadcasted or used in any way whatsoever directly or indirectly without the prior written consent of Vayyar or in any case of third party materials, without that third party's consent. This document shall be returned to Vayyar promptly upon request.

The materials and information contained herein are provided "AS IS" and Vayyar does not provide any warranties of any kind, whether express or implied, including but not limited to implied warranties of merchantability, fitness for a particular purpose, and non-infringement

Revision History

Revision	Date	Author	Description
1	July 2018	Yuval Mekamel	First issue
2	April 2019	Yuval Mekamel	Point Cloud Tracker
3	October 2019	Idan Ralbag	v6.0.0
4	May 2020	Yuval Mekamel	v6.1.0 + minor fixes
5	July 2020	Yuval Mekamel	v6.2.X
6	March 2021	Noam Reuveni	v7.0.0
7	July 2021	Noam Reuveni	v7.1.0

Table of Contents

CHAPTER 1 INTRODUCTION 4

1.1 Audience 4
1.2 Related Materials 4
1.3 Glossary 4

CHAPTER 2 OVERVIEW 5

2.1 System Architecture 5

CHAPTER 3 SESSION FLOW 7

3.1 Flow Overview 7
3.2 Detailed Flow 7

CHAPTER 4 SENSOR PARAMETERS 10

4.1 Arena Dimensions and Target Detection 10
4.2 File Management Settings 11

CHAPTER 5 SETTING OUTPUTS 13

5.1 Tracker Data Outputs 13
5.2 Point Cloud Outputs 15
5.3 IQ Data Outputs 17
5.4 Timestamp Outputs 17

CHAPTER 6 REFERENCE CODE 18

APPENDIX A VAYYAR'S WEB SOCKET PROTOCOL 20

Chapter 1
Introduction

This application note describes how to manage Vayyar RF sensors using the Vayyar API.

NOTE

Various programming languages can be used to produce software that will interwork with the Vayyar API. The examples and reference code provided in this reference guide were written in Python.

1.1 Audience

This document is targeted at the following audiences:

- Product and development personnel who are interested in integrating Vayyar RF sensors into their systems.

1.2 Related Materials

Refer to the following documents for additional information:

- *Installation and Operation Guide*

1.3 Glossary

Table 1: Glossary

Term	Definition
AC	Alternating Current
API	Application Programming Interface
DC	Direct Current
DLL	Dynamic Link Library
EVK	Evaluation Kit
GUI	Graphical User Interface
IP	Internet Protocol. An IP address is a numerical identifier assigned to a computing device or node in a TCP/IP network. The address is used to locate and identify the node in communications with other nodes on the network.
MCU	Micro-Controller Unit
PCB	Printed Circuit Board
RF	Radio Frequency
SoC	System-on-chip

Chapter 2
Overview

The Vayyar Application Programming Interface (API) provides programmatic access to the Vayyar EVK Engine, allowing for integration of Vayyar's non-contact RF sensors into customer-developed systems. The Vayyar sensors and operational software are used to count and track multiple people ("*targets*") in a defined space ("*arena*").

2.1 System Architecture

The Vayyar system architecture and connectivity scheme is illustrated in the figure below.

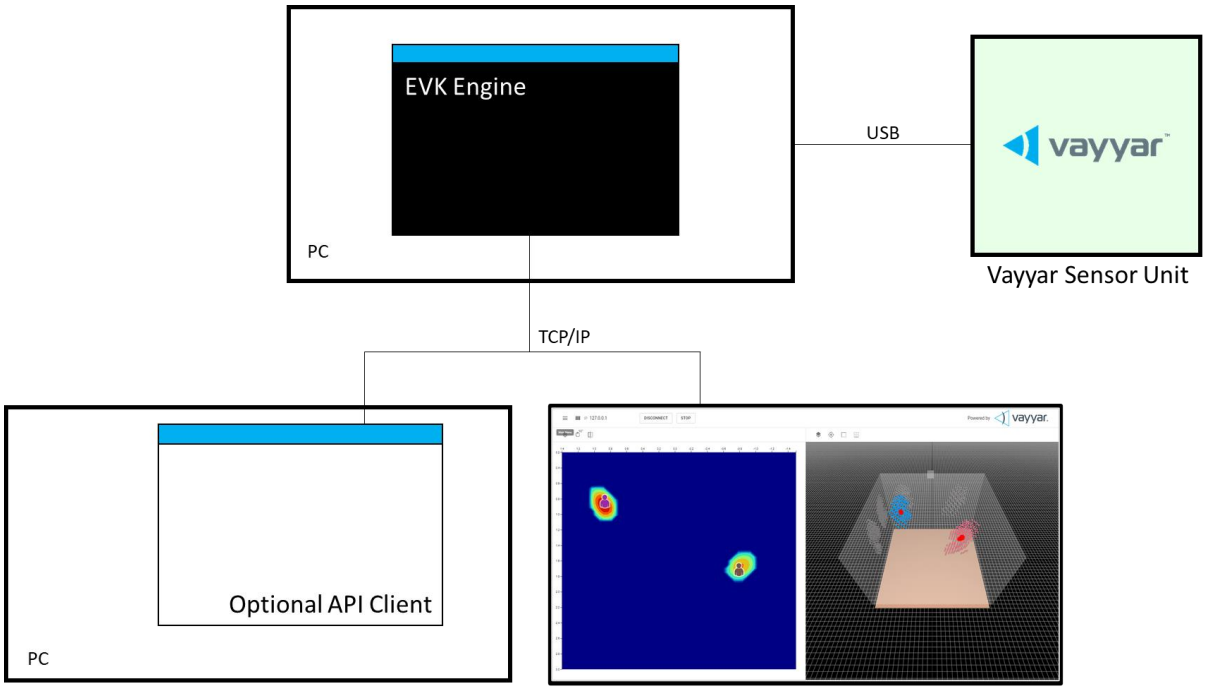


Figure 1: Vayyar System Architecture

The Vayyar solution is comprised of the following components:

- **RF-Based Sensor Unit.**
- **EVK Engine.** Host software that performs signal processing functions. The software runs in a command window in a PC, which is connected to the Vayyar Sensor via a USB cable.
- **Web GUI.** User-interface client software that communicates with the EVK Engine via a TCP/IP connection. The Web GUI client is supported by Google Chrome browser.
- **API Client.** Optional, customer-developed software that communicates with the EVK Engine via a TCP/IP connection.

NOTE

The Web GUI and API Client software can run either on their own dedicated PC, or on the same PC as the EVK Engine.

The Vayyar EVK Engine, which performs all backend calculations, creates a local server on the PC, using websocket technology. The API client then connects to the websocket server.

The websocket server runs on http://127.0.0.1:1234/, so a client running on the same PC needs to connect to the socket at this IP address/port. To connect from a different PC, the client should connect to ws://'EVK-PC-IP':1234/ ('EVK-PC-IP' = IP address of the PC to which EVK is connected). Note that the server connection is *not* secure, so using *wss://* instead of *ws://* is not recommended.

Messages are relayed in JSON format, so each message should be converted to JSON before sending and parsed when received. From the client perspective (probably JavaScript), a message is a simple JS object.

Chapter 3
Session Flow

This chapter describes how to initiate and manage a session with the Vayyar sensor via the API.

3.1 Flow Overview

1. Initialize the engine
2. Connect to the engine
3. Get/set parameters
4. Set outputs
5. Start the session
6. Request data from the sensor
   The requested data/outputs will be received per each frame.
7. Stop the session
8. Close the connection

3.2 Detailed Flow

NOTE

To learn more about communicating with the socket, please refer to Appendix A.

To set up and operate the system using the API:

1. **Initialize** the EVK Engine by executing the file *VayyarEVKEngine.exe* at C:\Vayyar_EVK_vX.X.X\EVK.
2. **Connect** the API client to the EVK Engine server over the web-socket connection:

```
listener = create_connection("ws://127.0.0.1:1234/")
```

3. **Set the sensor parameters** per the required configuration, as shown in the example below. To read the sensor parameters, simply change the message ID in the example to GET_PARAMS.

```
listener.send(json.dumps({
    'Type': 'COMMAND',
    'ID': 'SET_PARAMS',
    'Payload': {
        'Cfg.MonitoredRoomDims': [0, 0, 0, 0, 0, 0],
        'MPR.read_from_file': 0.0,
```

```
        'MPR.save_to_file': 0.0,
        'Cfg.OutputData.save_to_file': 0.0
    }}))
```

Refer to section 4.1 for detailed information on arena and target parameters.
Refer to section 4.2 for detailed information on file management parameters.

4. **Configure output settings**. Determine the types of data to be sent by the EVK Engine server, as shown in the example below.

```
# set outputs for each frame
    listener.send(json.dumps({
        'Type': 'COMMAND', 'ID': 'SET_OUTPUTS',
        'Payload': {
        'binary_outputs': ['points_Loc', 'targets_IDs'],
        'json_outputs': ['TimeStamp']
        }}))
```

Refer to Chapter 5 for detailed information on output settings.

5. **Start the session**, as shown in the example below. Once started, the requested outputs are being sent by the sensor.

```
listener.send(json.dumps({
        'Type': 'COMMAND',
        'ID': 'START',
        'Payload': {}
    }))
```

6. **Data request**. The EVK Engine server sends data messages on request from the client, which should send a data request following each frame to get the next one. Note that the server sends frame data only when new frame data is available – it will not send the same frame data twice.

```
    listener.send(json.dumps({'Type': 'QUERY', 'ID': 'BINARY_DATA'}))
    listener.send(json.dumps({'Type': 'QUERY', 'ID': 'JSON_DATA'}))
```

An example of a data message received from the server:

```
TimeStamp
18_03_2021_16_30_34.717
points_Locs
[[-0.05745243  0.33003906  1.2701903 ]
 [-0.01436311  0.34411557  1.2845534 ]
 [ 0.01436311  0.32436461  1.25582719]
 ...
 [-3.04976628  2.36931766  6.89574034]
 [-2.81516887  3.07671614  6.66114293]
 [-2.58057147  2.8732197   6.89574034]]
targets_IDs
[1. 5. 6. 8.]
```

Request status data. The EVK Engine server sends status messages on request from the client. To get the current status:

```
#send:
{
    'Type': 'COMMAND', 'ID': 'GET_STATUS'
}
#receive:
status = json.loads(client.recv())
print(status ['‘Payload’] ['status’])
```

The EVK GET_STATUS command will return one of the following:

- Initializing
- Calibration
- Imaging
- Configuring
- Error state

7. **End the session**, stops the sensor operation that was initiated by START command.

```
# stop the engine
    listener.send(json.dumps({
        'Type': 'COMMAND',
        'ID': 'STOP',
        'Payload': {}
    }))
```

8. **Close the connection**, terminates the web-socket connection between the API client and the sensor (server).

```
listener.close()
```

Chapter 4
Sensor Parameters

This chapter contains a description of sensor-related parameters that can be set or returned using the SET_PARAMS and GET_PARAMS calls, as shown in the example below:

```
listener.send(json.dumps({
        'Type': 'COMMAND',
        'ID': 'SET_PARAMS',
        'Payload': {
            'Cfg.MonitoredRoomDims': [-1.5, 1.5, 0.2, 3, 0.2, 1.8],
            'Cfg.Common.sensorOrientation.mountPlane': 'xz',
            'Cfg.Common.sensorOrientation.transVec(3)': 1.8,
            'Cfg.filterType': 'mti'
        }}))

listener.send(json.dumps({
    'Type': 'COMMAND',
    'ID': 'GET_PARAMS',
    'Payload': {
            'Cfg.MonitoredRoomDims': [0, 0, 0, 0, 0, 0]
         }}))
```

4.1 Arena Dimensions and Target Detection

The *Cfg.MonitoredRoomDims* parameter specifies arena dimensions in relation to the point of origin (the point on the floor directly below the sensor). Refer to orientation per sensor in the Arena Axes sections in the EVK User Guide.

The parameter values are designated in meters, according to the format: [Xmin, Xmax, Ymin, Ymax, Zmin, Zmax]. For example:

```
'Cfg.MonitoredRoomDims': [-1.5, 1.5, -2, 2, 0.0, 1.8],
```

The *Cfg.Common.sensorOrientation.mountPlane* parameter specifies the sensor orientation:

- 'xy' – parallel to the ceiling
- 'xz' – parallel to a wall

```
'Cfg.Common.sensorOrientation.mountPlane': 'xy',
```

The third value of the *Cfg.Common.sensorOrientation.transVec* parameter specifies the height of the sensor in meters, measured from the floor towards the center of the sensor – as in the **Sensor Height** field in the WebGUI

```
'Cfg.Common.sensorOrientation.transVec(3)': 1.8,
```

The *Cfg.filterType* parameter specifies the subtraction mode used by the sensor:

- **'no_subtraction': No Subtraction Mode.** Does not perform any arena background subtraction. Therefore, it displays static and moving targets.
- **'init_subtraction: Initial Image Subtraction Mode.** Records the arena for a few seconds when sensor starts (calibration mode) and subtracts the recorded data from each future recording. It shows changes in the arena from the initial state – including moving targets (such as people) and static targets (such as chairs that moved after the calibration).
- **'mti': Moving Target Indicator (MTI) Mode.** Shows only moving targets - displaying only the differences in the arena relative to previous few seconds of operation.

```
' Cfg.filterType': 'mti',
```

4.2 File Management Settings

The *MPR.save_dir* parameter specifies the folder in which the recordings will be saved and played from.

NOTE

The folder should be located on the same PC used to run the EVK Engine.

```
'MPR.save_dir': r'C:\path_of_directory',
```

The *MPR.read_from_file* parameter, when set to 1, allows playback of previously recorded files, without requiring a sensor. The data is played from the path specified in the *MPR.save_dir* field. Only raw data (.MAT) files can be played.

```
'MPR.read_from_file': 0,
```

The *MPR.allow_savedData_override* parameter, when set to 1, the system allows existing recording files, in MPR.save_dir, to be overwritten by new recording files.

```
'MPR.allow_savedData_override': 0,
```

The *MPR.save_to_file* parameter, when set to 1, saves ".MAT" files containing the raw data matrices in frequency domain together with timestamps. The files are saved in the path specified in the *MPR.save_dir* field.

```
'MPR.save_to_file': 0,
```

The *Cfg.OutputData.save_to_file* parameter, when set to 1, saves the locations – in [X Y Z] coordinates – of the tracked targets in each frame, together with timestamps. "NA" means no target was detected. The file is saved as .csv files in the path specified in the *MPR.save_dir* field in the ApplicationLogs directory.

```
'Cfg.OutputData.save_to_file': 0,
```

Chapter 5
Setting Outputs

This chapter contains a description of the data output settings of the EVK Engine server, as shown in the example below. Two types of data are transferred to the client:

- Binary output – useful for large transfers of raw data (I, Q) and 2D/3D mat. Binary output supports numerical data only.
- JSON output – used to transfer all types of data, but less efficient for large data transfers. JSON outputs should not be configured when the WebGUI (which utilizes the JSON outputs) is running.

```
# set outputs for each frame
    listener.send(json.dumps({
        'Type': 'COMMAND', 'ID': 'SET_OUTPUTS',
        'Payload': {
        'binary_outputs': ['points_Loc', 'targets_IDs'],
        'json_outputs': ['TimeStamp']
        }}))
```

NOTE

If the API client and the WebGUI are running in parallel, the SET_OUTPUTS command will override the settings configured using the WebGUI and might stuck the WebGUI.

5.1 Tracker Data Outputs

'targets_Locs'

Shape: [N,3], Type: Float.

2D Matrix for all the targets in the arena.

N rows: N = total number of targets. One row for each target.

3 columns: X, Y, Z

Each target in the matrix described by its location (Cartesian, in meters).

'targets_IDs'

Shape: [N,1], Type: Float.

2D Matrix for all the targets in the arena.

N rows: N = total number of targets. One row for each target.

1 column: ID of the relevant target.

Each target in the matrix described by its ID.

'targets_targetsCount'

Shape: [], Type: Float.

The number of targets detected in the arena.

'targets_Heights'

Shape: [N,1], Type: Float

2D Matrix for all the targets in the arena.

N rows: N = total number of targets. One row for each target.

1 column: Height of the relevant target

Each target in the matrix described by its height.

'targets_Velocities'

Shape: [N,2], Type: Float

2D Matrix for all the targets in the arena.

N rows: N = total number of targets. One row for each target.

2 columns: Speed in the X axis (m/s), Speed in the Y axis (m/s).

Each target in the matrix described by its velocity in the x and y axis.

'targets_Speeds'

Shape: [N,1], Type: Float

2D Matrix for all the targets in the arena.

N rows: N = total number of targets. One row for each target.

1 column: Speed of the targets in the arena in m/s, (absolute value of the velocity in the x and y axis).

Each target in the matrix described by its speed.

5.2 Point Cloud Outputs

'points_Locs'

Shape: [N,3], Type: Float.

2D Matrix for all the points in the arena.

N rows: N = total number of points. One row for each point.

3 columns: X, Y, Z

Each point in the matrix described by its location (Cartesian, in meters).

'points_absValues'

Shape: [N,1], Type: Float.

2D Matrix for all the points in the arena.

N rows: N = total number of points. One row for each point.

1 column: Intensity corresponding to the relevant point.

Each point in the matrix described by its intensity.

'points_realValues'

Shape: [N,1], Type: Float.

2D Matrix for all the points in the arena.

N rows: N= total number of points. One row for each point.

1 column: The real segment of the complex number that represents the voxel in the arena corresponding to the relevant point.

'points_imaginaryValues'

Shape: [N,1], Type: Float

2D Matrix for all the points in the arena

N rows: N= total number of points. One row for each point.

1 column: The imaginary segment of the complex number that represents the voxel in the arena corresponding to the relevant point.

'points_pointsCount'

Shape: [], Type: Float

Number of points detected in the arena.

'targetPoints'

Shape: [N,4], Type: Float.

2D Matrix for all the points in the arena.

N rows: N = total number of points. One row for each point.

4 columns: X, Y, Z, ID

Each point in the matrix described by its location (Cartesian, in meters) and the ID of the target it represents.

Note:
These points are derived from the "raw points" and associated to a specific target and based on the tracker algorithm. There might be frames where the target is tracked but no "targetPoints" are associated with the target.

5.3 IQ Data Outputs

'iqData_Smat_real', 'iqData_Smat_imaginary'

Shape: [pairs,freqs], Type: Float.

I/Q 2D matrix of Antenna Pairs x Frequency Points, calibrated and divided by reference. Each value in the matrix represents amplitude and phase measured in the relevant frequency and antennas. The values are complex numbers.

- 'iqData_Smat_real' - The real segment of the raw data in a 2D matrix.
- 'iqData_Smat_imaginary' - The imaginary segment of the raw data in a 2D matrix.

'iqData_AntPairs'

Shape: [pairs,2], Type: Float.

Transmit/receive antenna pairs of the I/Q data.

'iqData_Freqs'

Shape: [freqs,], Type: Float.

Recorded frequencies of the I/Q data.

5.4 Timestamp Outputs

'TimeStamp'

Shape: [dd_mm_yyyy_HH_MM_SS.FFF], Type: String

A string of the relevant timestamp per frame.

Chapter 6
Reference Code

NOTE

The reference code below, written in Python, requires Python 3.6.x or above.

```
from struct import unpack_from
import json
import numpy as np
from websocket import create_connection

DTYPES = {
    0: np.int8,
    1: np.uint8,
    2: np.int16,
    3: np.uint16,
    4: np.int32,
    5: np.uint32,
    6: np.float32,
    7: np.float64,
}

ASCII_RS = '\u001e'
ASCII_US = '\u001f'

def to_message(buffer):
    """ parse MatNet messages from JSON / Vayyar internal binary format """
    if isinstance(buffer, str):
        return json.loads(buffer)
    seek = 0
    fields_len = unpack_from('i', buffer, seek + 4)[0]
    fields_split = unpack_from(str(fields_len) + 's', buffer, seek +
8)[0].decode('utf8').split(ASCII_RS)
    msg = {'ID': fields_split[0], 'Payload':
dict.fromkeys(fields_split[1].split(ASCII_US))}
    seek += 8 + fields_len
    for key in msg['Payload']:
        seek += np.int32().nbytes
        dtype = DTYPES[np.frombuffer(buffer, np.int32, 1, seek).item()]
        seek += np.int32().nbytes
        ndims = np.frombuffer(buffer, np.int32, 1, seek).item()
        seek += np.int32().nbytes
        dims = np.frombuffer(buffer, np.int32, ndims, seek)
        seek += ndims * np.int32().nbytes
        data = np.frombuffer(buffer, dtype, np.prod(dims), seek)
        seek += np.prod(dims) * dtype().nbytes
        msg['Payload'][key] = data.reshape(dims) if ndims else data.item()
    return msg


def main():
    # """ connect to server and echoing messages """
    listener = create_connection("ws://127.0.0.1:1234/")
    # retrieve current configuration
    listener.send(json.dumps({
        'Type': 'COMMAND',
        'ID': 'SET_PARAMS',
        'Payload': {
```

```
            'Cfg.MonitoredRoomDims': [-2, 2, 0.5, 3, 0.2, 2],# Room dimensions
            'Cfg.Common.sensorOrientation.mountPlane':'xz',  #xy - ceiling, xz - wall
            'Cfg.Common.sensorOrientation.transVec(3)': 1.8, # Height of sensor
            'Cfg.filterType':'mti', #'mti', 'no_subtraction', 'init_subtraction'
            'MPR.save_dir': r'C:\recording_path',      # Saved records directory
            'MPR.read_from_file': 0.0,                 # 1 - To play records
            'MPR.allow_savedData_override': 0.0,       # 1 - To override existing recording
            'MPR.save_to_file ': 0.0,                   # 1 - To save rawMemory data
            'MPR.save_image_to_file': 0.0,             # 1 - To save image data
            'Cfg.OutputData.save_to_file': 0.0,        # 1 - To save log data
        }
    }))


    # set outputs for each frame
    listener.send(json.dumps({
        'Type': 'COMMAND',
        'ID': 'SET_OUTPUTS',
        'Payload': {
         'binary_outputs': [' targets_Locs', 'targets_IDs', 'targets_targetsCount',
'targets_Heights', 'targets_Velocities', 'targets_Speeds', 'points_Locs',
'points_absValues', 'points_realValues', ' points_imaginaryValues', 'points_pointsCount,
'iqData_Smat_real', 'iqData_Smat_imaginary', 'iqData_Freqs', 'iqData_AntPairs',
'targetPoints', ],
         'json_outputs': ['TimeStamp']
        }
    }))

    # Start the engine - only if WebGUI isn't present
    listener.send(json.dumps({
        'Type': 'COMMAND',
        'ID': 'START',
        'Payload': {}
    }))

    # request for binary and json data.
    listener.send(json.dumps({'Type': 'QUERY', 'ID': 'BINARY_DATA'}))
    listener.send(json.dumps({'Type': 'QUERY', 'ID': 'JSON_DATA'}))
    print("Running! Waiting for messages...")
    while True:
        buffer = listener.recv()
        data = to_message(buffer)
        print(data['ID'])
        if data['ID'] == 'BINARY_DATA':
            for key, item in enumerate(data['Payload']):
                print(item)
                print(data['Payload'][item])
            listener.send(json.dumps({'Type': 'QUERY', 'ID': 'BINARY_DATA'}))
        if data['ID'] == 'JSON_DATA':
            for key, item in enumerate(data['Payload']):
                print(item)
                print(data['Payload'][item])
            listener.send(json.dumps({'Type': 'QUERY', 'ID': 'JSON_DATA'}))
        if data['ID'] == 'GET_STATUS':
            print(data['Payload']['status'])

    listener.close()


if __name__ == '__main__':
    main()
```

Appendix A
Vayyar's Web Socket Protocol

The communication using the WebSocket (getting/setting parameters, starting a new session, etc.) is done with the *socket.send(…)* method. This method sends JSON formatted messages using the WebSocket object it is called on.

NOTE

In our example code, we create a WebSocket object (named *listener*) using the *create_connection* function.

Available Attributes for the JSON messages:

- ***'Type'*** – type of the message. Possible values:
  - *'COMMAND'* – request for performing an action, such as getting parameters or starting a new session. For 'COMMAND' messages that require additional data to be sent, the data will be specified in the 'Payload' attribute (see below).
    For example, for requesting to set parameters it is required to send the parameters themselves. These will be specified in the 'Payload' attribute.
  - *'QUERY'* – request for getting data, such as binary data or json data out of the operated session.
- ***'ID'*** – name of the message. Possible values:
  - *'SET_PARAMS'*
  - *'SET_OUTPUTS'*
  - *'START'*
  - *'STOP'*
  - *'BINARY_DATA'*
  - *'JSON_DATA'*
  - *'GET_STATUS'*
- ***'Payload'*** – the actual intended message. Examples for possible values:
  - *{ 'Cfg.MonitoredRoomDims': [-2, 2, 0.2, 4, 0.2, 1.8] }* –
    specifying arena dimensions to set.
  - *{ 'binary_outputs': [' points_Locs', 'freqs'], 'json_outputs': ['TimeStamp'] }* – specifying output types to get.

Example for sending a 'SET_PARAMS' message to the socket:

NOTE

The *json.dumps(…)* method serializes 'obj' to a JSON formatted 'str'.

```
listener.send(json.dumps({
    'Type': 'COMMAND',
    'ID': 'SET_PARAMS',
    'Payload': {
# Room dimensions:
        'Cfg.MonitoredRoomDims': [-0.1, 2, 0.5, 3, 0.2, 1.8],
# The plane of the sensor. xy - ceiling, xz - wall:
        'Cfg.Common.sensorOrientation.mountPlane': 'xz',
# The height of the sensor:
        'Cfg.Common.sensorOrientation.transVec(3)': 1.8,
# The filter used – 'mti', 'no_subtraction', 'init_subtraction':
        'Cfg.filterType': 'mti',
# The path where the recordings will be saved and played from:
        'MPR.save_dir': r'',
# 1 - Play records:
        'MPR.read_from_file': 0.0,
# 1 – Allows to override existing recording:
        'MPR. allow_savedData_override': 0.0,
# 1 - Save raw data:
        'MPR.save_to_file': 0.0,
# 1 - Save log data; Will be save under .\ApplicationLogs:
        'Cfg.OutputData.save_to_file': 0.0
}}))
```

Example for sending a 'BINARY_DATA' message to the socket:

```
listener.send(json.dumps({ 'Type': 'QUERY', 'ID': 'BINARY_DATA' }))
```

END OF DOCUMENT